

DESIGN AND ANALYSIS OF ALGORITHMS

DAY 6 – DYNAMIC PROGRAMMING – GRAPH ALGORITHMS

Name: N. PARNITA

Register No.: 192525322

Course code: CSA0617

Course Name: Design and Analysis of Algorithm

TABLE OF CONTENTS

- Experiment 1: Bellman-Ford Algorithm
- Experiment 2: Floyd-Warshall Algorithm
- Experiment 3: Warshall's Algorithm
- Experiment 4: City with the Smallest Number of Reachable Cities
- Experiment 5: Network Delay Time
- Experiment 6: Cheapest Flight Within K Stops
- Experiment 7: Detect Negative Weight Cycle
- Experiment 8: Determine Reachability Between Two Vertices
- Experiment 9: Count Reachable Vertices from a Source
- Experiment 10: Diameter of a Graph
- Experiment 11: Shortest Distance Between Two Vertices
- Experiment 12: Vertex Reachable from Every Other Vertex
- Experiment 13: Vertex with Maximum Reachability
- Experiment 14: Diameter of a Weighted Graph
- Experiment 15: Check Whether a Graph is Strongly Connected

Experiment 1: Bellman-Ford Algorithm

Objective: To find the shortest distance from a source vertex to all other vertices in a weighted directed graph and detect a negative weight cycle.

Problem Statement: To find the shortest distance from a source vertex to all other vertices in a weighted directed graph and detect a negative weight cycle.

Sample Input: Vertices = 5; Edges = [(0,1,-1),(0,2,4),(1,2,3),(1,3,2),(1,4,2),(3,2,5),(3,1,1),(4,3,-3)];
Source = 0

Sample Output: 0: 0, 1: -1, 2: 2, 3: -2, 4: 1

Time Complexity: $O(VE)$ time and $O(V)$ space.

Explanation: Bellman-Ford repeatedly relaxes all directed edges. After $V-1$ passes, one additional pass is used to detect a reachable negative-weight cycle.

Python Program

```
def bellman_ford(vertices, edges, source):
    dist = [float("inf")] * vertices
    dist[source] = 0

    for _ in range(vertices - 1):
        updated = False
        for u, v, weight in edges:
            if dist[u] != float("inf") and dist[u] + weight < dist[v]:
                dist[v] = dist[u] + weight
                updated = True
        if not updated:
            break

    for u, v, weight in edges:
        if dist[u] != float("inf") and dist[u] + weight < dist[v]:
            return None

    return dist

vertices = int(input("Enter number of vertices: "))
m = int(input("Enter number of edges: "))

edges = []
for _ in range(m):
    u, v, w = map(int, input("Enter u v weight: ").split())
    edges.append((u, v, w))

source = int(input("Enter source: "))
dist = bellman_ford(vertices, edges, source)

if dist is None:
    print("Negative Weight Cycle Detected")
else:
    for i, d in enumerate(dist):
        print(i, d)
```

Execution / Output

```
Enter number of vertices: 5
Enter number of edges: 8
Enter u v weight: 0 1 -1
Enter u v weight: 0 2 4
Enter u v weight: 1 2 3
Enter u v weight: 1 3 2
Enter u v weight: 1 4 2
Enter u v weight: 3 2 5
Enter u v weight: 3 1 1
Enter u v weight: 4 3 -3
Enter source: 0
0 0
1 -1
2 2
3 -2
4 1
```

Result: The program was implemented and the expected result was obtained.

Experiment 2: Floyd-Warshall Algorithm

Objective: To find the shortest distances between every pair of vertices in a weighted graph represented by an adjacency matrix.

Problem Statement: To find the shortest distances between every pair of vertices in a weighted graph represented by an adjacency matrix.

Sample Input: Graph = `[[0,5,INF,10],[INF,0,3,INF],[INF,INF,0,1],[INF,INF,INF,0]]`

Sample Output: `[[0,5,8,9],[INF,0,3,4],[INF,INF,0,1],[INF,INF,INF,0]]`

Time Complexity: $O(V^3)$ time and $O(V^2)$ space.

Explanation: Floyd-Warshall progressively allows each vertex to act as an intermediate vertex and updates all-pairs shortest distances.

Python Program

```
INF = float("inf")

def floyd_warshall(graph):
    n = len(graph)
    dist = [row[:] for row in graph]

    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] != INF and dist[k][j] != INF:
                    dist[i][j] = min(
                        dist[i][j],
                        dist[i][k] + dist[k][j]
                    )

    return dist

n = int(input("Enter number of vertices: "))
graph = []

print("Enter the adjacency matrix (use INF for no edge):")
for _ in range(n):
    row = input().split()
    graph.append([
        INF if x.upper() == "INF" else int(x)
        for x in row
    ])

result = floyd_warshall(graph)

for row in result:
    print(row)
```

Execution / Output

```
Enter number of vertices: 4
0 5 INF 10
INF 0 3 INF
INF INF 0 1
INF INF INF 0
```

```
[0, 5, 8, 9]  
[inf, 0, 3, 4]  
[inf, inf, 0, 1]  
[inf, inf, inf, 0]
```

Result: The program was implemented and the expected result was obtained.

Experiment 3: Warshall's Algorithm

Objective: To compute the transitive closure of a directed graph and determine whether a path exists between every pair of vertices.

Problem Statement: To compute the transitive closure of a directed graph and determine whether a path exists between every pair of vertices.

Sample Input: Graph = `[[0,1,0,0],[0,0,1,0],[0,0,0,1],[0,0,0,0]]`

Sample Output: `[[0,1,1,1],[0,0,1,1],[0,0,0,1],[0,0,0,0]]`

Time Complexity: $O(V^3)$ time and $O(V^2)$ space.

Explanation: Warshall's algorithm computes transitive closure by determining whether a path exists between every pair of vertices through intermediate vertices.

Python Program

```
def warshall(graph):
    n = len(graph)
    reach = [row[:] for row in graph]

    for k in range(n):
        for i in range(n):
            for j in range(n):
                reach[i][j] = (
                    reach[i][j] or
                    (reach[i][k] and reach[k][j])
                )
    return reach

n = int(input("Enter number of vertices: "))
graph = []
for _ in range(n):
    graph.append(list(map(int, input().split())))

result = warshall(graph)

for row in result:
    print(row)
```

Execution / Output

```
Enter number of vertices: 4
0 1 0 0
0 0 1 0
0 0 0 1
0 0 0 0
[0, 1, 1, 1]
[0, 0, 1, 1]
[0, 0, 0, 1]
[0, 0, 0, 0]
```

Result: The program was implemented and the expected result was obtained.

Experiment 4: City with the Smallest Number of Reachable Cities

Objective: To find the city that can reach the smallest number of other cities within a given distance threshold using Floyd-Warshall.

Problem Statement: To find the city that can reach the smallest number of other cities within a given distance threshold using Floyd-Warshall.

Sample Input: $n = 4$; edges = $[[0,1,3],[1,2,1],[2,3,4],[0,3,7]]$; threshold = 4

Sample Output: 3

Time Complexity: $O(V^3)$ time and $O(V^2)$ space.

Explanation: Floyd-Warshall progressively allows each vertex to act as an intermediate vertex and updates all-pairs shortest distances.

Python Program

```
INF = float("inf")

def find_city(n, edges, threshold):
    dist = [[INF] * n for _ in range(n)]

    for i in range(n):
        dist[i][i] = 0

    for u, v, w in edges:
        dist[u][v] = w
        dist[v][u] = w

    for k in range(n):
        for i in range(n):
            for j in range(n):
                dist[i][j] = min(
                    dist[i][j],
                    dist[i][k] + dist[k][j]
                )

    best_city = -1
    best_count = INF

    for i in range(n):
        count = sum(
            1 for j in range(n)
            if i != j and dist[i][j] <= threshold
        )

        if count <= best_count:
            best_count = count
            best_city = i

    return best_city

n = int(input("Enter number of cities: "))
m = int(input("Enter number of edges: "))
edges = [tuple(map(int, input().split())) for _ in range(m)]
threshold = int(input("Enter threshold: "))
```



```
print(find_city(n, edges, threshold))
```

Execution / Output

```
Enter number of cities: 4
Enter number of edges: 4
0 1 3
1 2 1
2 3 4
0 3 7
Enter threshold: 4
3
```

Result: The program was implemented and the expected result was obtained.

Experiment 5: Network Delay Time

Objective: To find the minimum time required for a signal to reach all nodes in a network using the Bellman-Ford algorithm.

Problem Statement: To find the minimum time required for a signal to reach all nodes in a network using the Bellman-Ford algorithm.

Sample Input: times = [[2,1,1],[2,3,1],[3,4,1]], n = 4, k = 2

Sample Output: 2

Time Complexity: $O(V \cdot E)$ time and $O(V)$ space.

Explanation: Bellman-Ford repeatedly relaxes all directed edges. After $V-1$ passes, one additional pass is used to detect a reachable negative-weight cycle.

Python Program

```
def network_delay(times, n, k):
    dist = [float("inf")] * (n + 1)
    dist[k] = 0
    for _ in range(n - 1):
        updated = False
        for u, v, w in times:
            if dist[u] != float("inf") and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                updated = True
        if not updated:
            break
    answer = max(dist[1:])
    return -1 if answer == float("inf") else answer

n = int(input("Enter number of nodes: "))
m = int(input("Enter number of edges: "))

times = []
for _ in range(m):
    times.append(list(map(int, input("Enter u v time: ").split())))

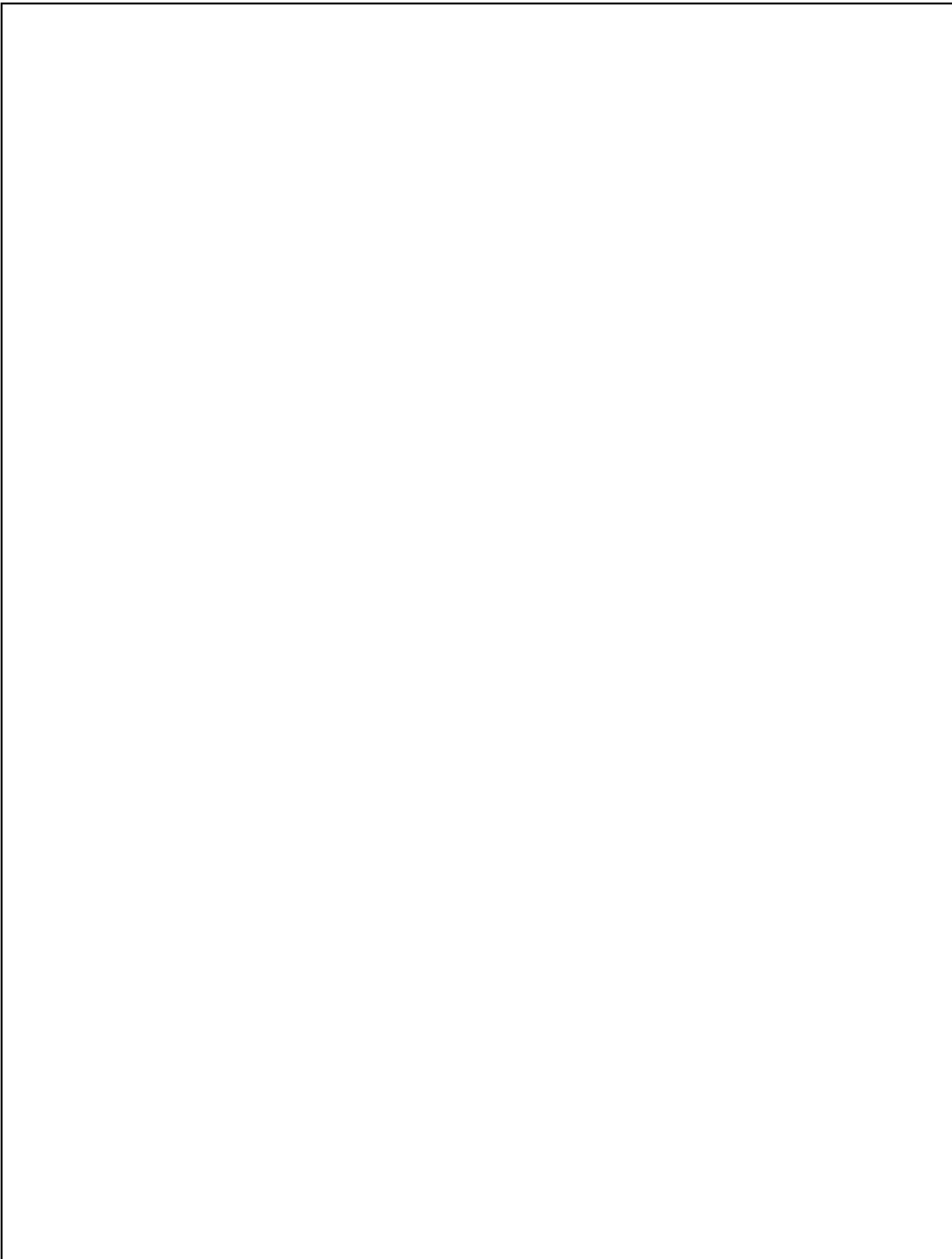
k = int(input("Enter source node: "))

print(network_delay(times, n, k))
```

Execution / Output

```
Enter number of nodes: 4
Enter number of edges: 3
2 1 1
2 3 1
3 4 1
Enter source node: 2
2
```

Result: The program was implemented and the expected result was obtained.



Experiment 6: Cheapest Flight Within K Stops

Objective: To find the minimum cost flight from a source city to a destination with at most K stops using Bellman-Ford.

Problem Statement: To find the minimum cost flight from a source city to a destination with at most K stops using Bellman-Ford.

Sample Input: $n = 4$; flights = [[0,1,100],[1,2,100],[2,3,100],[0,3,500]]; src = 0; dst = 3; K = 2

Sample Output: 300

Time Complexity: $O(K \cdot E)$ time and $O(V)$ space.

Explanation: Bellman-Ford repeatedly relaxes all directed edges. After $V-1$ passes, one additional pass is used to detect a reachable negative-weight cycle.

Python Program

```
def cheapest_flight(n, flights, src, dst, K):
    dist = [float("inf")] * n
    dist[src] = 0

    for _ in range(K + 1):
        new_dist = dist[:]

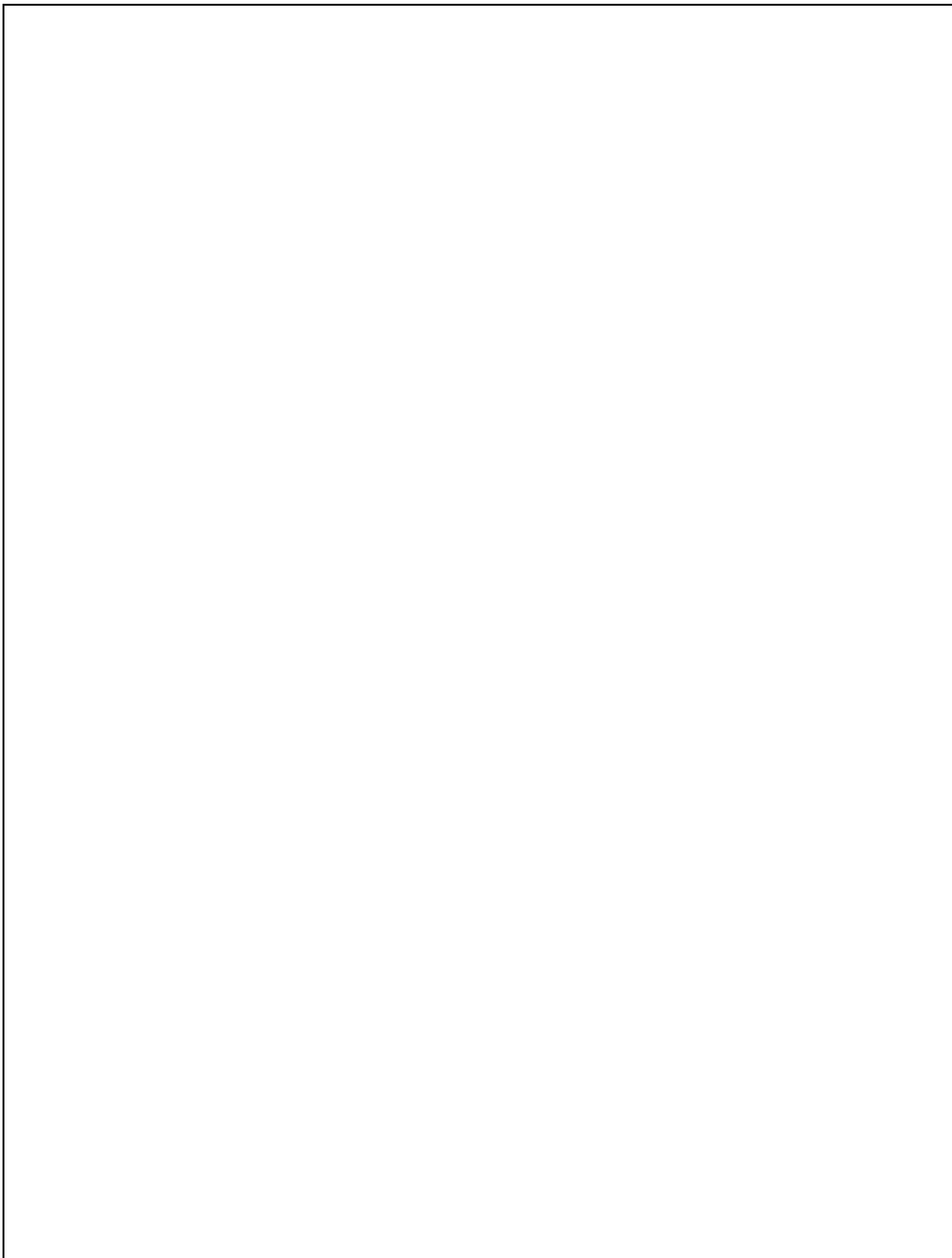
        for u, v, cost in flights:
            if dist[u] != float("inf"):
                new_dist[v] = min(
                    new_dist[v],
                    dist[u] + cost
                )
        dist = new_dist
    return -1 if dist[dst] == float("inf") else dist[dst]

n = int(input("Enter number of cities: "))
m = int(input("Enter number of flights: "))
flights = [list(map(int, input().split())) for _ in range(m)]
src = int(input("Enter source: "))
dst = int(input("Enter destination: "))
K = int(input("Enter maximum stops: "))
print(cheapest_flight(n, flights, src, dst, K))
```

Execution / Output

```
Enter number of cities: 4
Enter number of flights: 4
0 1 100
1 2 100
2 3 100
0 3 500
Enter source: 0
Enter destination: 3
Enter maximum stops: 2
300
```

Result: The program was implemented and the expected result was obtained.



Experiment 7: Detect Negative Weight Cycle

Objective: To determine whether a weighted directed graph contains a negative weight cycle using Bellman-Ford.

Problem Statement: To determine whether a weighted directed graph contains a negative weight cycle using Bellman-Ford.

Sample Input: Vertices = 4; Edges = [(0,1,1),(1,2,-1),(2,3,-1),(3,0,-1)]

Sample Output: Negative Weight Cycle Exists

Time Complexity: $O(VE)$ time and $O(V)$ space.

Explanation: Bellman-Ford repeatedly relaxes all directed edges. After $V-1$ passes, one additional pass is used to detect a reachable negative-weight cycle.

Python Program

```
def has_negative_cycle(vertices, edges):
    dist = [0] * vertices

    for _ in range(vertices - 1):
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w

    for u, v, w in edges:
        if dist[u] + w < dist[v]:
            return True

    return False

vertices = int(input("Enter number of vertices: "))
m = int(input("Enter number of edges: "))

edges = []
for _ in range(m):
    edges.append(tuple(map(int, input("Enter u v weight: ").split())))

if has_negative_cycle(vertices, edges):
    print("Negative Weight Cycle Exists")
else:
    print("No Negative Weight Cycle")
```

Execution / Output

```
Enter number of vertices: 4
Enter number of edges: 4
0 1 1
1 2 -1
2 3 -1
3 0 -1
Negative Weight Cycle Exists
```

Result: The program was implemented and the expected result was obtained.

Experiment 8: Determine Reachability Between Two Vertices

Objective: To determine whether a path exists from vertex u to vertex v using Warshall's transitive closure.

Problem Statement: To determine whether a path exists from vertex u to vertex v using Warshall's transitive closure.

Sample Input: Graph = $[[0,1,0],[0,0,1],[0,0,0]]$, $u = 0$, $v = 2$

Sample Output: Path Exists

Time Complexity: $O(V^3)$ time and $O(V^2)$ space.

Explanation: Warshall's algorithm computes transitive closure by determining whether a path exists between every pair of vertices through intermediate vertices.

Python Program

```
def warshall(graph):
    n = len(graph)
    reach = [row[:] for row in graph]

    for i in range(n):
        reach[i][i] = 1

    for k in range(n):
        for i in range(n):
            for j in range(n):
                reach[i][j] = (
                    reach[i][j] or
                    (reach[i][k] and reach[k][j])
                )
    return reach

n = int(input("Enter number of vertices: "))
graph = [list(map(int, input().split())) for _ in range(n)]
u = int(input("Enter u: "))
v = int(input("Enter v: "))
reach = warshall(graph)
if reach[u][v]:
    print("Path Exists")
else:
    print("Path Does Not Exist")
```

Execution / Output

```
Enter number of vertices: 3
0 1 0
0 0 1
0 0 0
Enter u: 0
Enter v: 2
Path Exists
```

Result: The program was implemented and the expected result was obtained.

Experiment 9: Count Reachable Vertices from a Source

Objective: To find the number of vertices reachable from a specified source using Warshall's transitive closure, excluding the source itself.

Problem Statement: To find the number of vertices reachable from a specified source using Warshall's transitive closure, excluding the source itself.

Sample Input: Graph = $[[0,1,0,0],[0,0,1,0],[0,0,0,1],[0,0,0,0]]$, source = 0

Sample Output: 3

Time Complexity: $O(V^3)$ time and $O(V^2)$ space.

Explanation: Warshall's algorithm computes transitive closure by determining whether a path exists between every pair of vertices through intermediate vertices.

Python Program

```
def count_reachable(graph, source):
    n = len(graph)
    reach = [row[:] for row in graph]

    for k in range(n):
        for i in range(n):
            for j in range(n):
                reach[i][j] = (
                    reach[i][j] or
                    (reach[i][k] and reach[k][j])
                )

    return sum(
        1 for j in range(n)
        if j != source and reach[source][j]
    )

n = int(input("Enter number of vertices: "))
graph = [list(map(int, input().split())) for _ in range(n)]
source = int(input("Enter source: "))

print("Reachable vertices =", count_reachable(graph, source))
```

Execution / Output

```
Enter number of vertices: 4
0 1 0 0
0 0 1 0
0 0 0 1
0 0 0 0
Enter source: 0
Reachable vertices = 3
```

Result: The program was implemented and the expected result was obtained.

Experiment 10: Diameter of a Graph

Objective: To find the maximum shortest-path distance between any pair of vertices using Floyd-Warshall.

Problem Statement: To find the maximum shortest-path distance between any pair of vertices using Floyd-Warshall.

Sample Input: Graph = [[0,2,INF],[2,0,3],[INF,3,0]]

Sample Output: 5

Time Complexity: $O(V^3)$ time and $O(V^2)$ space.

Explanation: Floyd-Warshall progressively allows each vertex to act as an intermediate vertex and updates all-pairs shortest distances.

Python Program

```
INF = float("inf")

def graph_diameter(graph):
    n = len(graph)
    dist = [row[:] for row in graph]

    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] != INF and dist[k][j] != INF:
                    dist[i][j] = min(
                        dist[i][j],
                        dist[i][k] + dist[k][j]
                    )

    diameter = 0

    for i in range(n):
        for j in range(n):
            if dist[i][j] != INF:
                diameter = max(diameter, dist[i][j])

    return diameter

n = int(input("Enter number of vertices: "))
graph = []

for _ in range(n):
    row = input().split()
    graph.append([
        INF if x.upper() == "INF" else int(x)
        for x in row
    ])

print("Diameter =", graph_diameter(graph))
```

Execution / Output

```
Enter number of vertices: 3
0 2 INF
```

```
2 0 3  
INF 3 0  
Diameter = 5
```

Result: The program was implemented and the expected result was obtained.

Experiment 11: Shortest Distance Between Two Vertices

Objective: To find the shortest distance between two vertices using Floyd-Warshall.

Problem Statement: To find the shortest distance between two vertices using Floyd-Warshall.

Sample Input: Graph = [[0,3,INF,7],[8,0,2,INF],[5,INF,0,1],[2,INF,INF,0]], u = 0, v = 3

Sample Output: 6

Time Complexity: $O(V^3)$ time and $O(V^2)$ space.

Explanation: Floyd-Warshall progressively allows each vertex to act as an intermediate vertex and updates all-pairs shortest distances.

Python Program

```
INF = float("inf")

def shortest_distance(graph, u, v):
    n = len(graph)
    dist = [row[:] for row in graph]

    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] != INF and dist[k][j] != INF:
                    dist[i][j] = min(
                        dist[i][j],
                        dist[i][k] + dist[k][j]
                    )
    return dist[u][v]

n = int(input("Enter number of vertices: "))
graph = []
for _ in range(n):
    row = input().split()
    graph.append([
        INF if x.upper() == "INF" else int(x)
        for x in row
    ])

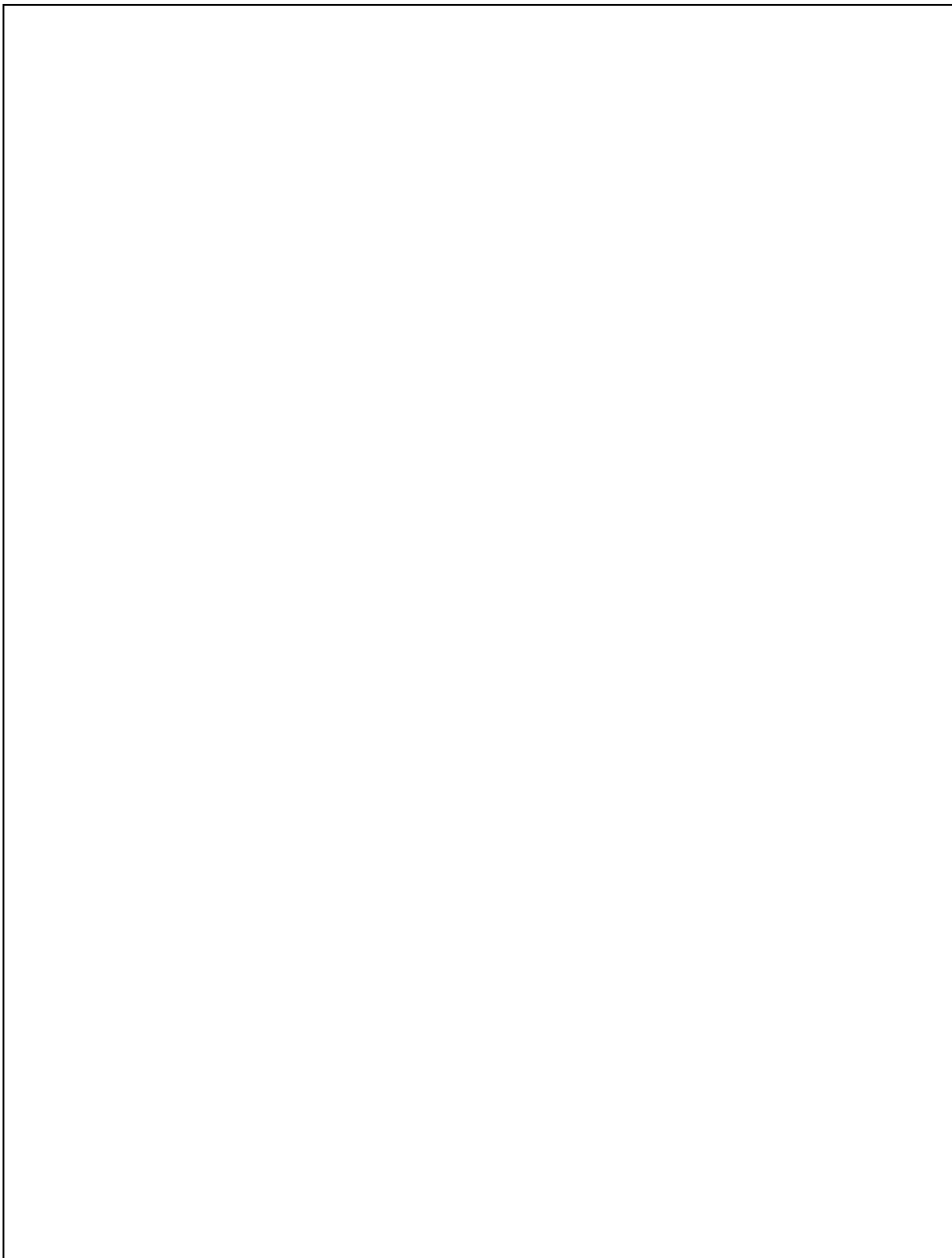
u = int(input("Enter u: "))
v = int(input("Enter v: "))

print("Shortest Distance =", shortest_distance(graph, u, v))
```

Execution / Output

```
Enter number of vertices: 4
0 3 INF 7
8 0 2 INF
5 INF 0 1
2 INF INF 0
Enter u: 0
Enter v: 3
Shortest Distance = 6
```

Result: The program was implemented and the expected result was obtained.



Experiment 12: Vertex Reachable from Every Other Vertex

Objective: To find the smallest-indexed vertex that is reachable from every other vertex using Warshall's Algorithm.

Problem Statement: To find the smallest-indexed vertex that is reachable from every other vertex using Warshall's Algorithm.

Sample Input: Graph = [[0,1,1],[0,0,1],[0,0,0]]

Sample Output: 2

Time Complexity: $O(V^3)$ time and $O(V^2)$ space.

Explanation: Warshall's algorithm computes transitive closure by determining whether a path exists between every pair of vertices through intermediate vertices.

Python Program

```
def reachable_from_all(graph):
    n = len(graph)
    reach = [row[:] for row in graph]

    for i in range(n):
        reach[i][i] = 1

    for k in range(n):
        for i in range(n):
            for j in range(n):
                reach[i][j] = (
                    reach[i][j] or
                    (reach[i][k] and reach[k][j])
                )

    for v in range(n):
        if all(reach[u][v] for u in range(n)):
            return v

    return -1

n = int(input("Enter number of vertices: "))
graph = [list(map(int, input().split())) for _ in range(n)]

print("Vertex =", reachable_from_all(graph))
```

Execution / Output

```
Enter number of vertices: 3
0 1 1
0 0 1
0 0 0
Vertex = 2
```

Result: The program was implemented and the expected result was obtained.

Experiment 13: Vertex with Maximum Reachability

Objective: To find the vertex that can reach the maximum number of other vertices using Warshall's Algorithm.

Problem Statement: To find the vertex that can reach the maximum number of other vertices using Warshall's Algorithm.

Sample Input: Graph = [[0,1,1,0],[0,0,1,0],[0,0,0,1],[0,0,0,0]]

Sample Output: 0

Time Complexity: $O(V^3)$ time and $O(V^2)$ space.

Explanation: Warshall's algorithm computes transitive closure by determining whether a path exists between every pair of vertices through intermediate vertices.

Python Program

```
def max_reachability(graph):
    n = len(graph)
    reach = [row[:] for row in graph]

    for k in range(n):
        for i in range(n):
            for j in range(n):
                reach[i][j] = (
                    reach[i][j] or
                    (reach[i][k] and reach[k][j])
                )

    best_vertex = 0
    best_count = -1

    for i in range(n):
        count = sum(reach[i])
        if count > best_count:
            best_count = count
            best_vertex = i

    return best_vertex

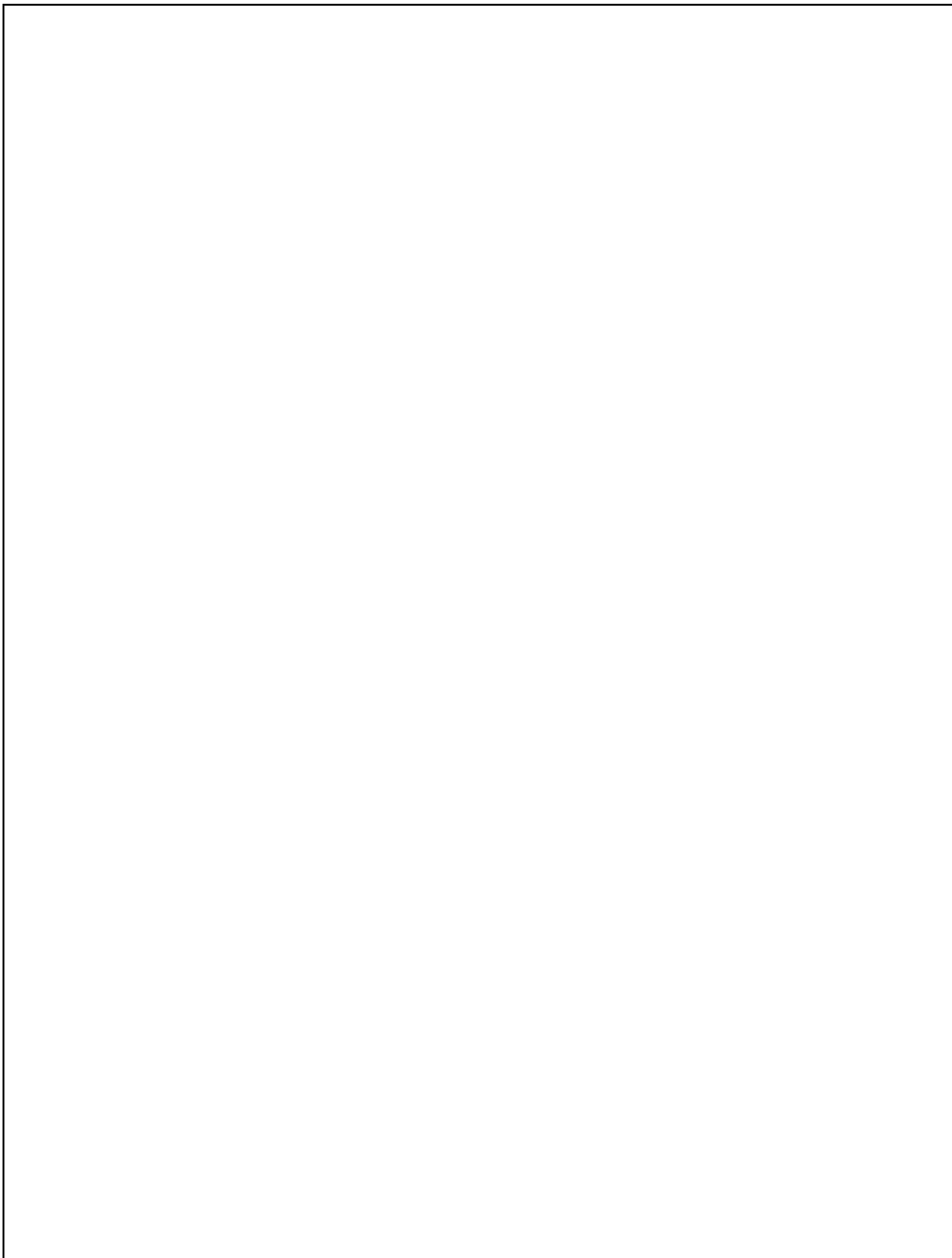
n = int(input("Enter number of vertices: "))
graph = [list(map(int, input().split())) for _ in range(n)]

print("Vertex with maximum reachability =", max_reachability(graph))
```

Execution / Output

```
Enter number of vertices: 4
0 1 1 0
0 0 1 0
0 0 0 1
0 0 0 0
Vertex with maximum reachability = 0
```

Result: The program was implemented and the expected result was obtained.



Experiment 14: Diameter of a Weighted Graph

Objective: To find the diameter of a weighted graph using Floyd-Warshall, where the diameter is the maximum shortest-path distance.

Problem Statement: To find the diameter of a weighted graph using Floyd-Warshall, where the diameter is the maximum shortest-path distance.

Sample Input: Graph = `[[0,2,INF],[2,0,4],[INF,4,0]]`

Sample Output: 6

Time Complexity: $O(V^3)$ time and $O(V^2)$ space.

Explanation: Floyd-Warshall progressively allows each vertex to act as an intermediate vertex and updates all-pairs shortest distances.

Python Program

```
INF = float("inf")

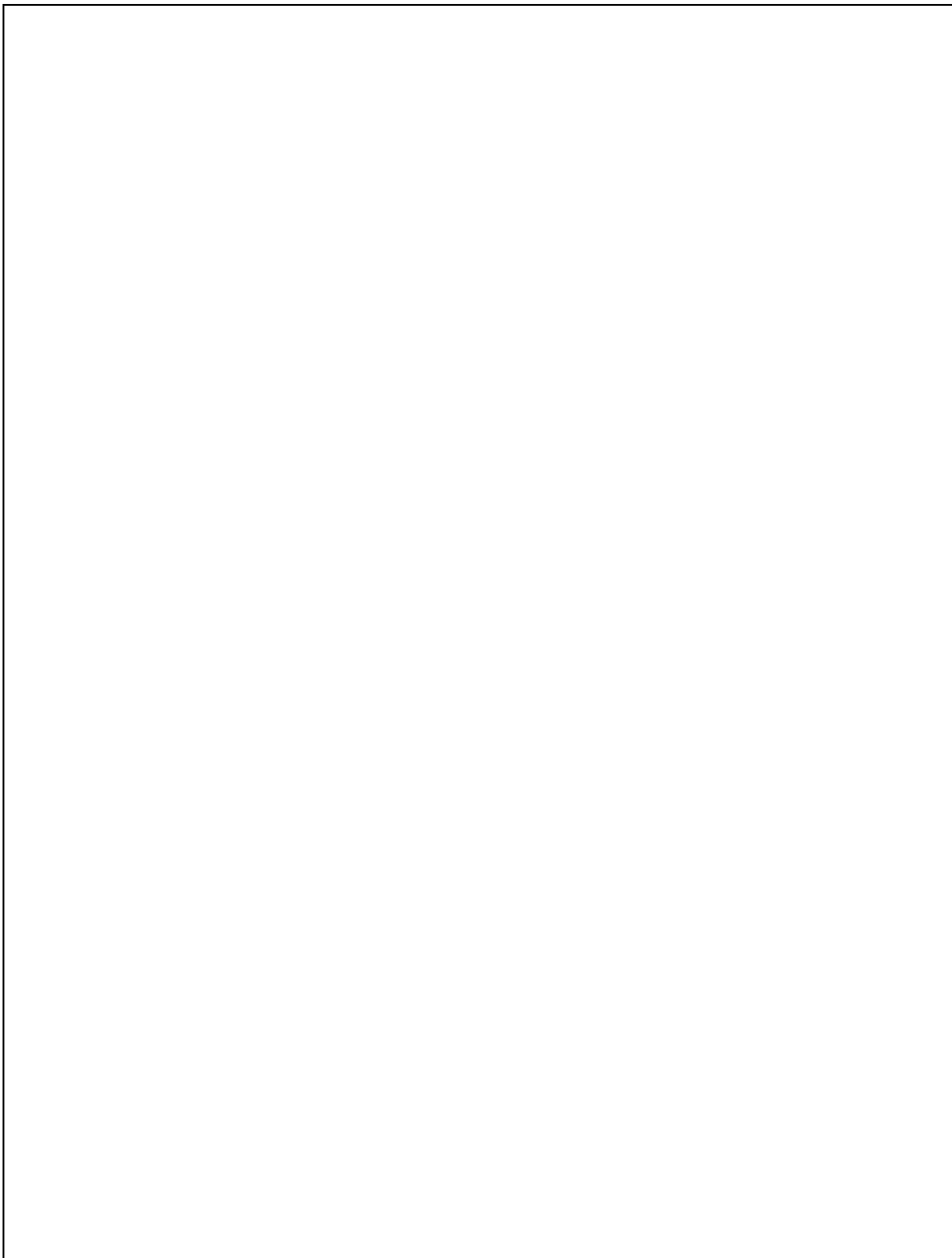
def weighted_diameter(graph):
    n = len(graph)
    dist = [row[:] for row in graph]
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] != INF and dist[k][j] != INF:
                    dist[i][j] = min(
                        dist[i][j],
                        dist[i][k] + dist[k][j]
                    )
    diameter = 0
    for i in range(n):
        for j in range(n):
            if dist[i][j] != INF:
                diameter = max(diameter, dist[i][j])
    return diameter

n = int(input("Enter number of vertices: "))
graph = []
for _ in range(n):
    row = input().split()
    graph.append([
        INF if x.upper() == "INF" else int(x)
        for x in row
    ])
print("Diameter =", weighted_diameter(graph))
```

Execution / Output

```
Enter number of vertices: 3
0 2 INF
2 0 4
INF 4 0
Diameter = 6
```

Result: The program was implemented and the expected result was obtained.



Experiment 15: Check Whether a Graph is Strongly Connected

Objective: To determine whether every vertex in a directed graph is reachable from every other vertex using Warshall's Algorithm.

Problem Statement: To determine whether every vertex in a directed graph is reachable from every other vertex using Warshall's Algorithm.

Sample Input: Graph = [[0,1,0],[0,0,1],[1,0,0]]

Sample Output: Strongly Connected

Time Complexity: $O(V^3)$ time and $O(V^2)$ space.

Explanation: Warshall's algorithm computes transitive closure by determining whether a path exists between every pair of vertices through intermediate vertices.

Python Program

```
def strongly_connected(graph):
    n = len(graph)
    reach = [row[:] for row in graph]

    for i in range(n):
        reach[i][i] = 1

    for k in range(n):
        for i in range(n):
            for j in range(n):
                reach[i][j] = (
                    reach[i][j] or
                    (reach[i][k] and reach[k][j])
                )

    return all(reach[i][j] for i in range(n) for j in range(n))

n = int(input("Enter number of vertices: "))
graph = [list(map(int, input().split())) for _ in range(n)]

if strongly_connected(graph):
    print("Strongly Connected")
else:
    print("Not Strongly Connected")
```

Execution / Output

```
Enter number of vertices: 3
0 1 0
0 0 1
1 0 0
Strongly Connected
```

Result: The program was implemented and the expected result was obtained.